
Travaux Pratiques

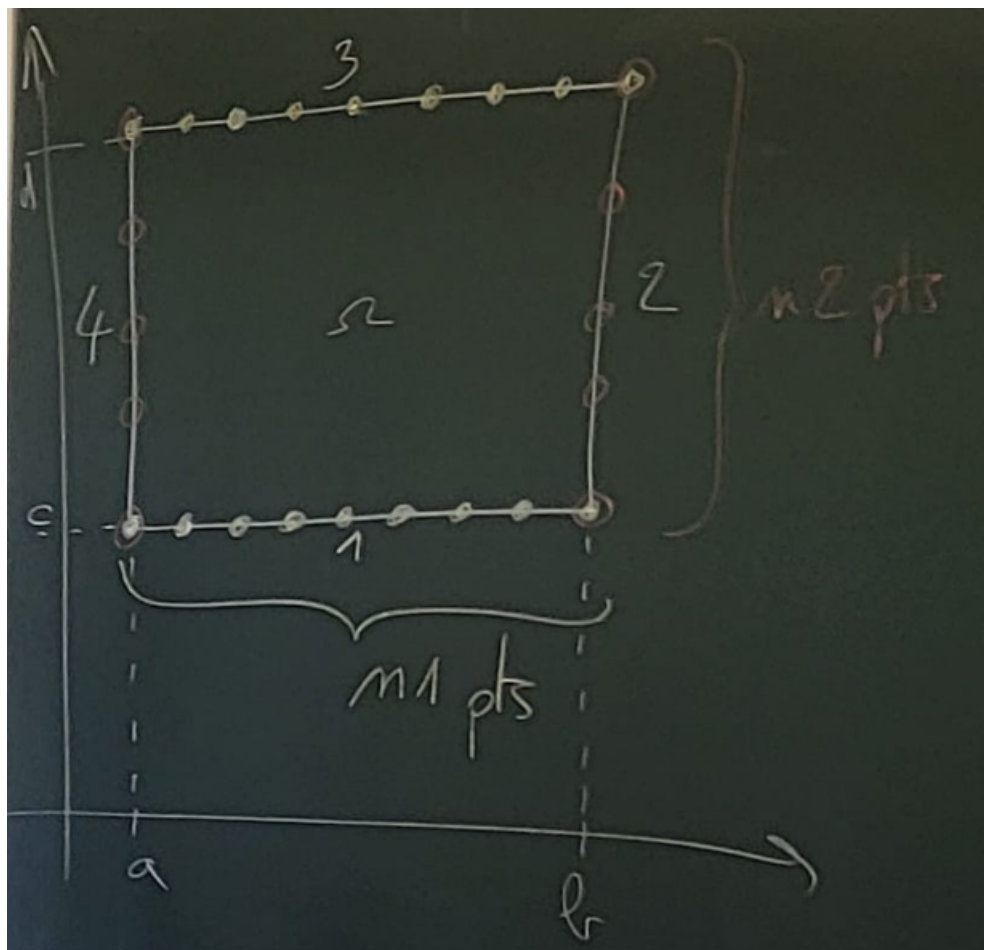
Éléments finis

Enseignant :
M. Eric DARRIGAND

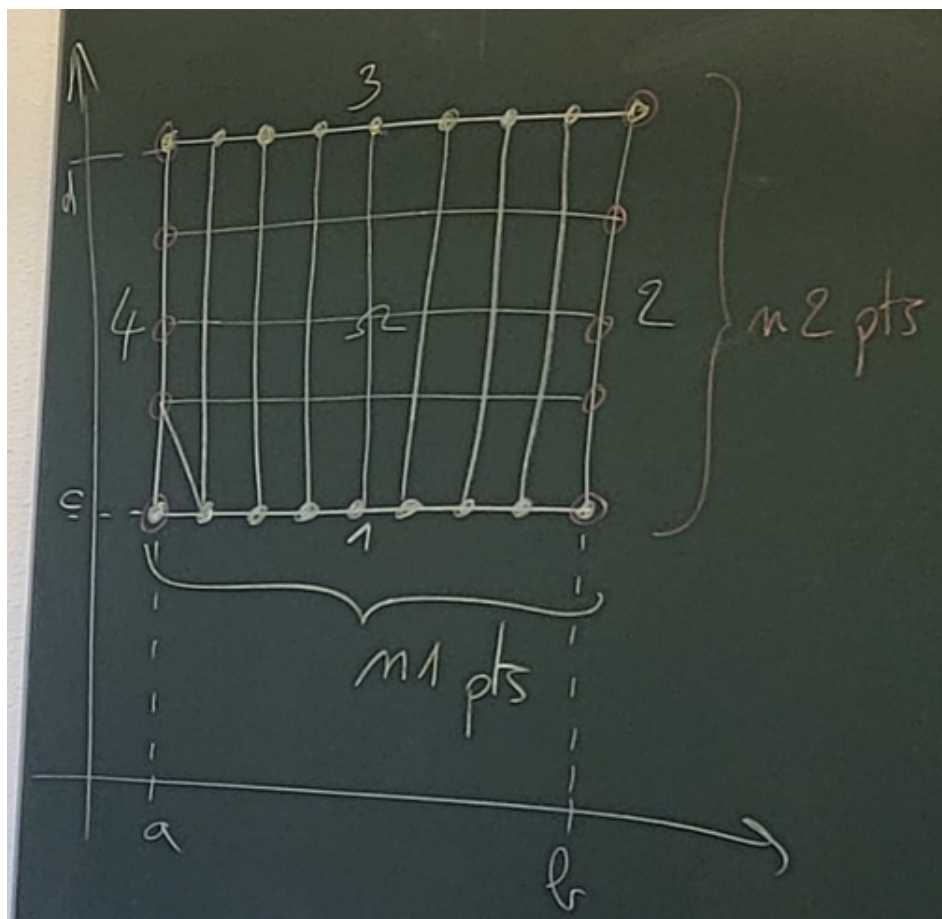
Étudiant :
Nathan HASCOET

28 janvier 2026

1 TP séance 1



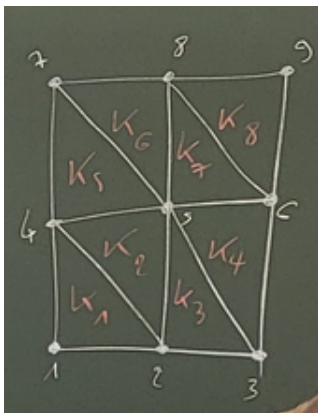
On construit le maillage (ici des rectangles) :



,pour des triangles, il suffit de rajouter les diagonales, voir sujet TP 1

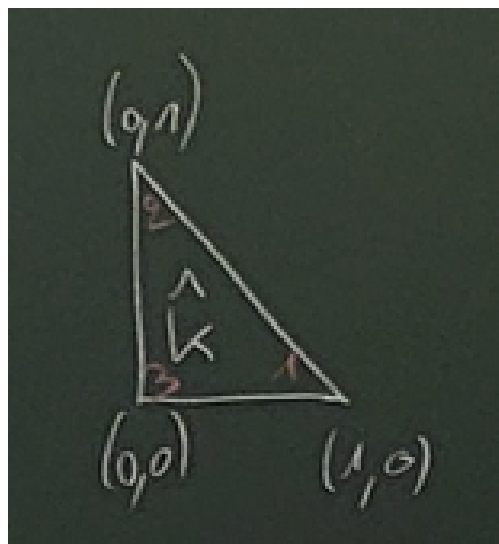
Maillage :

1. n : nombre de nœuds
2. $x_1 \ y_1$: coordonnées du nœud 1
3. $x_2 \ y_2$: coordonnées du nœud 2
- $\vdots$
- $x_n \ y_n$: coordonnées du nœud n
4. m : nombre d'éléments (triangles ou quadrilatères ici)
- t : type des éléments
- p : nombre de nœuds par éléments
5. $s_1^{(1)}, s_2^{(1)}, \dots, s_p^{(1)}$: liste des numéraux globaux des nœuds de l'élément 1.
- $\vdots$
- $s_1^{(m)}, s_2^{(m)}, \dots, s_p^{(m)}$: liste des numéraux globaux des nœuds de l'élément m .



6ème élément : 5 7 8

La **convention** que l'on adoptera pour la numérotation est la suivante :



le numéro local $\boxed{3}$ est celui de l'angle droit
et
et ensuite nous tournons en sens direct

1.1 Production perso

```
#include <stdio.h>
#include <stdlib.h>

void Creer_maillage(double a, double b, double c, double d, int n1, int n2, int t, FILE *fp)
{
    /*
    ===== Cette ALGORITHME Créer un maillage sur un quadrangle =====
    Omega = [a;b] x [c; d] avec a < b et c < d
    */

    int n = n1 * n2; // n = est le nombre de noeuds
    fprintf(fp, "%d\n", n);

    double h1 = (b-a)/(n1-1);
    double h2 = (d-c)/(n2-1);

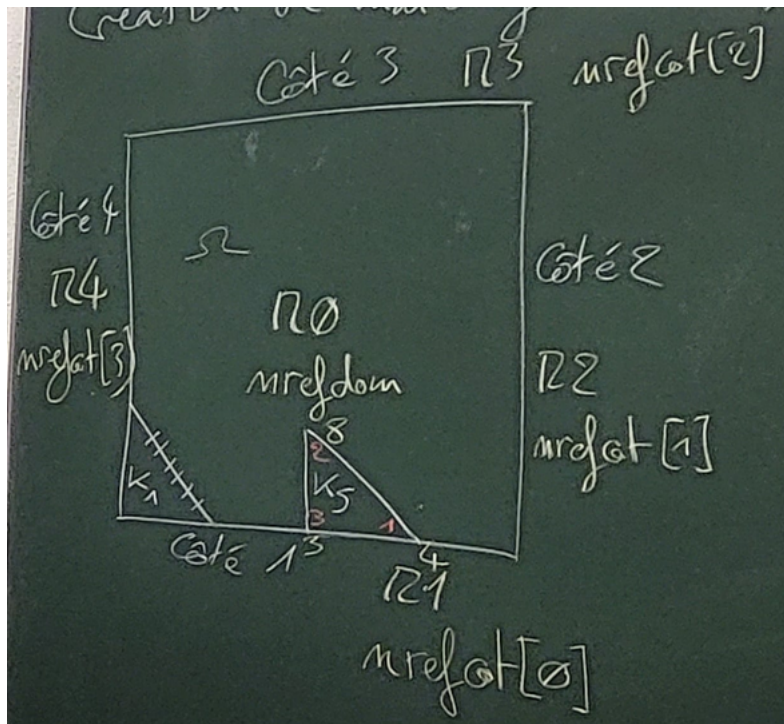
    double x,y;
    // Boucle sur les lignes (ordonnée)
    for (int i_y = 0; i_y < n1; i_y++)
    {
        // Boucle sur les colonnes (abscisse)
        for (int i_x = 0; i_x < n2; i_x++)
        {
            x = a + h1 * i_x;
            y = c + h2 * i_y;
            fprintf(fp, "(%lf,%lf)\n", x,y);
        }
    }

    int m = (n1 - 1)*(n2 - 1); // Nombre d'éléments (triangle ou quadrangle)
    int p;
    switch (t) {
        case 1 : // t = 1 : quadrangle
            p = 4; // nombre de noeuds par éléments
            fprintf(fp, "%d %d %d\n", m, t, p );
            for (int i = 0; i < (n1-2); i++) // boucle sur les lignes
            {
                for (int j = 0; j < (n2-2); j++) // boucle sur les colonnes
                {
                    //On récupère le numéro global du 1er sommet de l'élément (i+1)*(j+1)
                    int e0 = 2 + n1 * i + j ;
                    fprintf(fp, "%d %d %d %d\n", e0, e0 + n1, e0 + n1 - 1, e0 - 1);
                }
            }

        case 2 : // t = 2: triangle
            m = 2*m;
            p = 3;
            fprintf(fp, "%d %d %d\n", m, t, p );
            for (int i = 0; i < (n1-2); i++)
            {
                for (int j = 0; j < (n2-2); j++)
                {
                    //On récupère le numéro global du 1er sommet du triangle du bas
                    int e0 = 2 + n1 * i + j ;
                    // Triangle du bas
                    fprintf(fp, "%d %d %d\n", e0, e0 + n1, e0 - 1);
                    // Triangle du haut
                    fprintf(fp, "%d %d %d\n", e0 + n1 - 1, e0, e0 + n1);
                }
            }
    }
}
```

2 TP séance 2

Création du maillage :



★ Ecriture des coordonnées :

$$\begin{pmatrix} x_1 & y_1 \\ \vdots & \vdots \\ x_n & y_n \end{pmatrix}$$

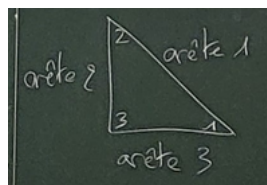
Coordonnée

→ Appel de **etiqAr** (calcul de **nRefAr**)

★ Ecriture des éléments :

$$\underbrace{\begin{pmatrix} s_1^{(1)} & \dots & s_p^{(1)} \\ \vdots & & \vdots \\ s_1^{(m)} & \dots & s_p^{(m)} \end{pmatrix}}_{\text{n gnel}} \quad \underbrace{\begin{pmatrix} r_1^{(1)} & \dots & r_q^{(1)} \\ \vdots & & \vdots \\ r_1^{(m)} & \dots & r_q^{(m)} \end{pmatrix}}_{\text{nRefAr}}$$

★ Numérotation locale :



Lecture du maillage

Ecrire une fonction *main* :

→ déclare des pointeurs : $\underbrace{\text{coord}}_{\text{float **}}, \underbrace{\text{n gnel}, \text{nRefAr}}_{\text{int **}}$

→ appel de *lecfima* (...,&coord,...) → allocation dynamique de coordonnée.

→ écrire le maillage lu dans un autre fichier

2.1 Production Perso

```
#include <stdio.h>
#include <stdlib.h>
#include "header.h"

void Creer_maillage(double a, double b, double c, double d, int n1, int n2, int t, FILE *fp)
{
    //===== Cette ALGORITHME Créer un maillage sur un quadrangle =====
    // Omega = [a;b] x [c; d] avec a < b et c < d
    // n1 = nombre de noeuds en abscisse
    // n2 = nombre de noeuds en ordonnée
    // t = type d'élément : t = 1 pour des quadrangles, t = 2 pour des triangles
    // fp = fichier de sortie

    int n = n1 * n2; // n = est le nombre de noeuds
    fprintf(fp, "%d\n", n);

    double h1 = (b-a)/(n1-1); //
    double h2 = (d-c)/(n2-1);

    double x,y;
    for (int i_y = 0; i_y < n2; i_y++) // Boucle sur les lignes (ordonnée)
    {
        for (int i_x = 0; i_x < n1; i_x++) // Boucle sur les colonnes (abscisse)
        {
            x = a + h1 * i_x;
            y = c + h2 * i_y;
            fprintf(fp, "%lf %lf\n", x, y);
        }
    }

    int m = (n1 - 1)*(n2 - 1); // Nombre d'éléments (triangle ou quadrangle)
    int *p_nbtel = &m; //nbtel = nombre d'éléments
    // REDONDANT, donné par m
    int p; // p = nombre de noeuds par élément
    int q; // q = nombre d'arêtes par élément
    // REDONDANT, donné par p, car c'est une figure fermée
    //On initialise le tableau des arrêtes
    int *p_nbaret = &p; // nbaret = nombre d'arrête dans un élément
    // REDONDANT, donné par p
    int nrefdom = 0; // l'intérieur du domaine Omega est numéroté : 0
    int *p_nRefAr, **nRefAr;
    const int nrefcot[] = {1,2,3,4}; // numérotation des bords du domaine Omega

    switch (t) {
        case 1 : // t = 1 : quadrangle
            p = 4; // p = 4 (nombre de noeuds par éléments)
            q = *p_nbaret;
            //On initialise le tableau contenant le numéro des arrêtes à 0
            p_nRefAr = malloc(*p_nbaret**p_nbtel*sizeof(*p_nRefAr));
            nRefAr = malloc(*p_nbtel*sizeof(*nRefAr));
            for (int i=0; i<*p_nbtel; i++) nRefAr[i] = p_nRefAr+i**p_nbaret;
            for (int i=0; i<*p_nbtel; i++) {
                for (int j=0; j<*p_nbaret; j++) {
                    nRefAr[i][j] = nrefdom;
                }
            }
        }

    etiqAr(t, n1, n2, nrefdom, nrefcot, *p_nbtel, *p_nbaret, nRefAr);

    fprintf(fp, "%d %d %d %d\n", m, t, p, q);
    for (int i = 0; i < (n1-1); i++) // boucle sur les colonnes
    {
```

```

for (int j = 0; j < (n2-1); j++) // boucle sur les lignes
{
    int e0 = 2 + n1 * i + j ;
    //On récupère le numéro global du 1er sommet de l'élément (i+1)*(j+1)
    int numero_element = j + i*(n1-1); // Numéro de l'élément sur lequel on est
    // e0 : sommet en bas à droite ===== e0+ n1 : sommet en haut à droite
    //e0 + n1 - 1 : sommet en haut à gauche ==== e0 - 1 : sommet en bas à gauche
    fprintf(fp,"%d %d %d %d %d %d %d %d\n",e0 + n1, e0 + n1 - 1, e0 - 1,
            nRefAr[numero_element][0],
            nRefAr[numero_element][1],
            nRefAr[numero_element][2],
            nRefAr[numero_element][3]);
}
}
break;

case 2 : // t = 2: triangle
m = 2*m;
p = 3; // p = 3
q = *p_nbaret;

//On initialise le tableau contenant le numéro des arrêtes à 0
p_nRefAr = malloc(*p_nbaret**p_nbtel*sizeof(*p_nRefAr));
nRefAr = malloc(*p_nbtel*sizeof(*nRefAr));
for (int i=0;i<*p_nbtel;i++) nRefAr[i] = p_nRefAr+i**p_nbaret;
for (int i=0;i<*p_nbtel;i++) {
    for (int j=0;j<*p_nbaret;j++) {
        nRefAr[i][j] = nrefdom;
    }
}

etiqaAr(t, n1, n2, nrefdom, nrefcot, *p_nbtel, *p_nbaret, nRefAr);

fprintf(fp,"%d %d %d %d\n",m, t, p, q);
for (int i = 0; i < (n1-1); i++) // boucle sur les colonnes
{
    for (int j = 0; j < (n2-1); j++) // boucle sur les lignes
    {
        int e0 = 2 + n1 * i + j ; //On récupère le numéro global du 1er sommet du triangle
        int numero_element = 2*(j + i*(n1-1)); // Numéro du triangle du bas
        fprintf(fp,"%d %d %d %d %d %d %d\n", e0, e0 + n1 - 1, e0 - 1,
                nRefAr[numero_element][0],
                nRefAr[numero_element][1],
                nRefAr[numero_element][2]);

        // Triangle du bas
        fprintf(fp,"%d %d %d %d %d %d\n", e0 + n1 - 1, e0, e0 + n1,
                nRefAr[numero_element+1][0],
                nRefAr[numero_element+1][1],
                nRefAr[numero_element+1][2]);

        // Triangle du haut
    }
}
break;
}
}

```



```

void etiqAr(int type1, int n1, int n2, int nrefdom, const int *nrefcot, int nbtel, int nbaret,
{
    /*
    n1 : nombre de sommets sur une ligne
    n2 : nombre de sommets sur une colonne

    type1 = 1 si les éléments du maillage sont des triangles et 2 si Carrés
    nrefdom = intérieur du domaine
    nrefcot = numéro des bords du maillage

    nRefAr = tableau de nbtel*nbaret lignes et 3 ou 4 colonnes (suivant si c'est un triangle)

    VARIABLES REDONDANTES !!
    nbtel = nombre d'éléments dans le maillage (dépend uniquement du type et de n1 et n2)
    nbaret = nombre d'arête dans un élément (dépend uniquement du type)
    */

    int c = (n1-1)*(n2-1)-(n1-2); // Numéro de l'élément en haut à gauche du maillage

    switch (type1) {
        //===== CARRE : énumération des arrêtes =====

        case 1 :
            //=====On numérote les éléments du bord=====
            //Ligne du bas du maillage
            for (int i = 0; i < n1-1; i++)
            {
                nRefAr[i][3] = nrefcot[0];
            }

            //Colonne de droite
            for (int i = n1-2; i < (n1-1)*(n2-1); i += n1-1)
            {
                nRefAr[i][0] = nrefcot[1];
            }

            //Ligne du haut du maillage
            for (int i = c-1; i < (n1-1)*(n2-1); i++)
            {
                nRefAr[i][1] = nrefcot[2];
            }

            //Colonne de gauche
            for (int i = 0; i < c; i += n1-1)
            {
                nRefAr[i][2] = nrefcot[3];
            }
            break;

        case 2 :
            //===== TRIANGLE : énumération des arrêtes =====
            //=====On numérote les éléments du bord=====
            //Ligne du bas du maillage (on saute un triangle sur 2)
            for (int i = 0; i < 2*(n1-1)-1; i += 2)
            {
                nRefAr[i][2] = nrefcot[0];
            }

            //Colonne de droite
            for (int i = 2*(n1-1)-1; i < 2*(n1-1)*(n2-1); i += 2*(n1-1))
            {
                nRefAr[i][1] = nrefcot[1];
            }

            //Ligne du haut du maillage
            for (int i = 2*c-1; i < 2*(n1-1)*(n2-1); i += 2)

```



```

{
    nRefAr[i][2] = nrefcot[2];
}

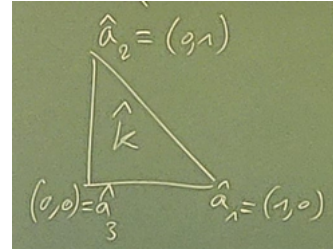
//Colonne de gauche
for (int i=0;i<2*c-1;i+=2*(n1-1))
{
    nRefAr[i][1] = nrefcot[3];
}
break;
}
}

```

3 TP séance 3

$$\int_K \nabla w_j^K \cdot \nabla w_i^K; \int_K w_j^K \cdot w_i^K; \int_K g_N \cdot w_i^K; \int_K f_\Omega \cdot w_i^K$$

Soit K' une arête d'un triangle, K dans $\mathcal{T}_h$ tel que $K' \subset \Gamma_{N,h}$.



$$\begin{aligned} \int_K a_{0,0} w_j^K w_i^K &= \int_{\widehat{K}} a_{0,0}(F_K(\hat{x})) \underbrace{w_j^K(F_K(\hat{x}))}_{=\hat{w}_j(\hat{x})} \underbrace{w_i^K(F_K(\hat{x}))}_{\hat{w}_i(\hat{x})} |\det JF_K(\hat{x})| d\hat{x} \\ &\simeq \sum_{k=1}^q \hat{\omega}_k a_{0,0}(F_K(\hat{x}_k)) \hat{\omega}_j(\hat{x}_k) \hat{\omega}_i(\hat{x}_k) |\det JF_K(\hat{x}_k)| \end{aligned}$$

Besoin de :

★ $\sum_k, \hat{\omega}_k, \hat{x}_k$ poids et points d'une formule de quadrature sur $\widehat{K}$.

★ $\hat{\omega}_i, i = 1, \dots, N^K$ sur $\widehat{K}$, **et leurs dérivées premières**.

★ F_K définie à partir des $\hat{\omega}_i$, $F_K(\hat{x}) = \sum_{i=1}^{N^K} \hat{\omega}_i(\hat{x}) q_i^K$.

JF_K s'obtient en dérivant l'expression précédente, et on en déduit **JF_K^{-1}**